

Course: DVA262 — Machine Learning Concepts

Objective: The purpose of the Lab is to apply your understanding of the k-NN algorithm by implementing the k-NN algorithm from scratch in the C++ programming using the Lab skeleton.

Lab 1: The k-NN Algorithm Implementation

Execution and demonstration

This lab is executed by a group of two students or individually. The completed lab should be demonstrated to a lab assistant.

Useful aids

- The course literature
- Lecture slides and notes
- Eigen <https://eigen.tuxfamily.org/dox/GettingStarted.html>
- Google & Internet
- Ask the lab assistant if anything is unclear

This lab is divided into two parts

- Part 1: k-NN for classification
- Part 2: k-NN for regression

To complete the lab, you need to implement and demonstrate the output of both parts.

Note:

1. Texts in the **Box** indicate functions that have already been implemented for the skeleton. You don't need to make any changes unless stated otherwise.
2. You should implement functions mentioned as "**Task**" in the **Your Task** sections.
3. Do not worry if the prediction accuracy is low. Discuss with the lab assistant.

Part 1: k-NN for Classification

You will implement the k-NN algorithm for the classification problem in this task.

Dataset: The classification problem will be based on the **Iris dataset**, which is already available in the project folder.

The Visual Interface: The main form (i.e., [MainForm.h](#)) calls all the class objects and functions that are already implemented in the skeleton. However, you may need to edit some parts of the code to make it consistent with your code to display the results correctly.

- *Loading and splitting the dataset*

The code for loading and preparing the dataset is already implemented, so no additional code is required for this part. Use the following function in your code:

- The function [loadAndPreprocessDataset\(\)](#) is used to prepare the dataset, which is located in the class name [DataLoader.cpp](#) under [DataUtils](#) folder.
- The [splitDataset\(\)](#) function in the class [DataPreprocessor.cpp](#) under the [DataUtils](#) folder is used to split the dataset into training and test sets.

Your Task: You will mainly work with two classes: [KNNClassifier.cpp](#) and [SimilarityFunctions.cpp](#), located under the [Classification](#) and [Utils](#) folder, respectively.

- *Creating class member functions*

The [KNNClassifier.cpp](#) class contains the following member functions:

- **Constructor:** initialises the member variables.
- **fit() function:** assigns training dataset and labels.
- **predict() function:** this is the main function that calculates the similarity between instances of test and training data sets and returns the classification prediction. **Note:** check the returns data type.

Note: You will be working on the [predict\(\)](#) function.

Task 1: In this Lab work, you must complete the [predict\(\)](#) function.

The [SimilarityFunctions.cpp](#) class contains several similarity function bodies which are not implemented.

Task 2: You must complete [euclideanDistance\(\)](#) function to compute the similarity between instances.

Task 3: In addition, you should at least implement one other distance function apart from the `eulideanDistance()` in the `SimilarityFunctions.cpp` class. Now, use that function for similarity measures in the `predict()` function. Does the algorithm perform the same when you use `eulideanDistance()`? Demonstrate and discuss the results with the lab assistant.

- *Evaluating*

When you complete the implementation, it is time to evaluate your algorithm by measuring accuracy, precision, recall and F1 score. The code for these metrics is already done, so no additional implementation is required.

- The functions `accuracy()`, `precision()`, `recall()` and `f1Score()` are used for model evaluation and they are located in the class name `Metrics.cpp` under the `Evaluation` folder.

Task 4: What value of k have you used? Try different values for k . What is the largest and smallest value of k you can choose? Discuss your findings during the demonstration.

Note: The value of k is set when you create an object of the `KNNClassifier.cpp` class in the `MainForm.cpp` class.

- *Visualisation:*

A visual representation (for training and test data sets) is needed. The code for this task is already done, so use the following functions:

- The function `confusionMatrix()` is used to calculate the confusion matrix for the specified number of class labels. The function is located in the class name `Metrics.cpp` under the `Evaluation` folder,
- The function `DisplayConfusionMatrixInGrid()` is used to Display the confusion matrix in the grid function and it is located in the `MainForm.h`.

An example of the result of k-NN classification is shown in Figure 1. The visual interface is the same for all classifiers.

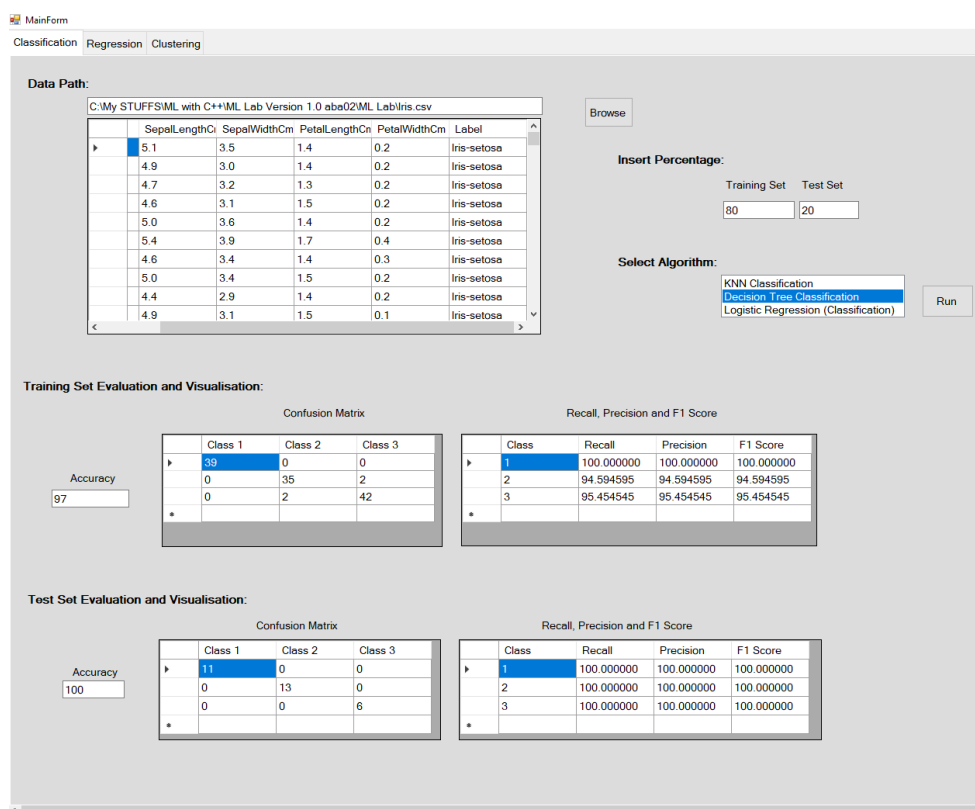


Figure 1: Example result of Decision Tree classification shown in the visual interface. The k-NN algorithm uses the same visual interface.

Part 2: k-NN for Regression

In this task, you will implement the k-NN algorithm for the regression problem.

Dataset: The regression problem will be based on the **Boston Housing** dataset, which is already available in the project folder.

The Visual Interface: Similar to the classification task, the main form (i.e., [MainForm.h](#)) calls all the class objects and functions that are already implemented. But you may need to edit some parts of the code to make it consistent with your code to display the results correctly.

- *Loading and splitting the dataset*

The code for loading and preparing (preprocessing) the dataset is already done, so no additional code is required for this part. Use the following function in your code:

- The function [readDatasetFromFilePath\(\)](#) is used to prepare the dataset, which is located in the class name [DataLoader.cpp](#) under [DataUtils](#) folder.
- The function [splitDataset\(\)](#) in the class name [DataPreprocessor.cpp](#) under the [DataUtils](#) folder is used to split the dataset into training and test sets.

Your Task: You will mainly work with the [KNNRegression.cpp](#) class located under the [Regression](#) folder. If you have already completed the **Part 1** then you do not need to implement anything more in the [SimilarityFunctions.cpp](#) class.

- *Creating class member functions*

The [KNNRegression.cpp](#) class contains the following member functions:

- [Constructor](#): initialises the number of neighbours
- [fit\(\) function](#): assigns training dataset and labels.
- [predict\(\) function](#): this is the main function that calculates the similarity between instance of test and training data set and returns the prediction. *Note*: check the returns data type.

Note: You will be working on the [predict\(\)](#) function.

Task 5: In this part of the work you must complete the [predict\(\)](#) function.

Task 6: If you have already completed **Task 2** and **Task 3**, you are good to go with the [SimilarityFunctions.cpp](#) class. Use the similarity function that you implemented in **Task 3** and show the result.

- Evaluating

Now, you need to evaluate the performance of the regression model measuring Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and R-Squared. The code for these metrics is already implemented.

- The functions `meanAbsoluteError()`, `rootMeanSquaredError()`, and `rSquared()` are used for model evaluation and they are located in the class name `Metrics` under the `Evaluation` folder.

Task 7: What value of k have you used? Try different values for k . What is the largest and smallest value of k you can choose? Discuss your findings during the demonstration.

Note: The value of k is set when you create an object of the `KNNRegression.cpp` class in the `MainForm.cpp` class.

- Visualisation

A visual representation (for training and test sets) is needed. The code for this task is already done, so use the following functions for that:

- The `VisualizeParityPlot()` function is used to display the parity plot for the actual and predicted test labels, which located in the `MainForm.h`,

An example of the result from the k -NN regression is shown in the Figure 2. The visual interface is same for all regression algorithms.

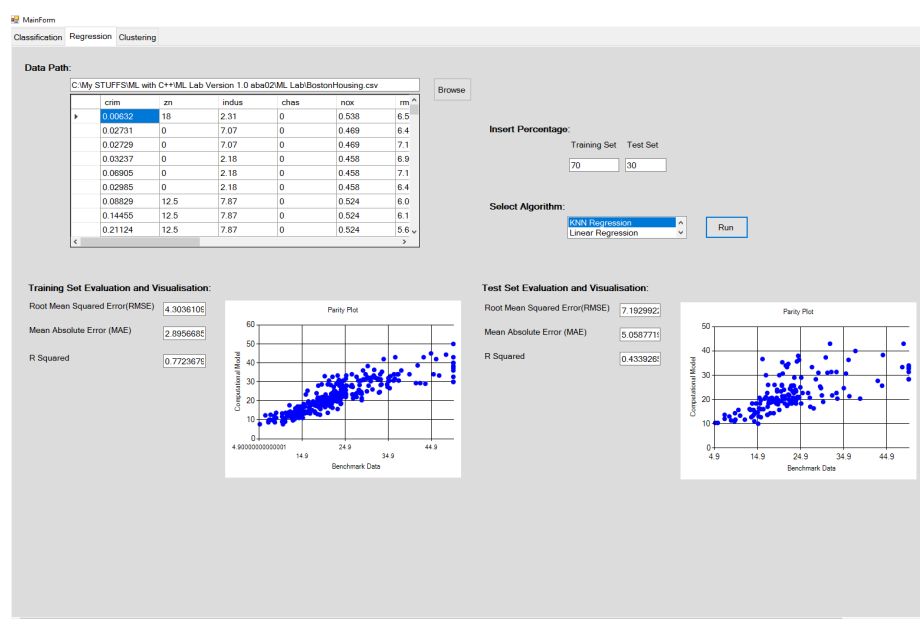


Figure 2: Example results of k -NN Regression in the visual interface.

Some Hints:

- Include the necessary header files at the beginning of your code.
- See lecture slides to predict labels considering k-nearest neighbours.
- Calculate the distance between the test data point and each training data point.
- Store predicted labels for all test data points.

Extra Assignment – if you want to learn some more.

Implement all the similarity functions in the [SimilarityFunctions.cpp](#) class. Test the performance of k-NN algorithm for both Classification and Regression using each similarity function in the [predict\(\)](#) function. Which similarity function works best?